COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Samarth Bhatt | UT EID: skb3295 | Repository:
github.com/skb3295/coe379L-midterm-Samarth-Bhatt

1. System Overview

When a user makes a request, the system receives a LabRequest. It holds the logged-in user plus any details that were already figured out ahead of time (project, resource, start and end time). Anything that wasn't figured out is left empty. The prompt sends this trusted information separately from the user's original message, and it tells the model that search results and tool outputs are untrusted data, not instructions. It also tells the model what it has to check before reserving anything, that it can only cite passages it actually retrieved, and that it can only say an action worked if the tool reported success.

The main loop in run_agent runs for at most 10 steps. At each step, the model has to return one of three typed decisions: a tool call, a clarification question, or a final answer. If its output doesn't match one of these, the run stops right away. A clarification goes straight back to the user without running any tools. For a tool call, the action gate runs first. It checks that the call still matches the original request (same user, project, resource, and times), and for a reservation it also checks that every required policy condition is met. If the gate says no, the tool never runs, and the model gets a denied result instead. Every attempt, allowed or not, is saved in the trace and sent back to the model for its next step. When the model gives a final answer, it is checked against the trace, and if it claims something that didn't actually happen, the run is marked as failed instead. If the model runs out of steps, the run also fails.

2. Public Benchmark Results

Table 1. Aggregate metrics from public_benchmark.json.

Metric	Value	Metric	Value
scenario_count	6	unnecessary_refusal_rate	0
success_rate	1.0	clarification_accuracy	1.0
invalid_structured_response_rate	0	average_tool_calls	2.67
unsafe_action_attempt_rate	0.0		

All six public scenarios passed. In the successful tensile tester case, the agent searched the policy, checked Alice's authorizations and the machine's status, made the reservation, and returned "completed" with a citation it had actually retrieved. The four blocked cases (missing laser training, expired calibration on the robot cell, no after-hours approval, and a time conflict) each ran three lookups and then stopped without trying to reserve. The ambiguous request asked for clarification without calling any tools. One interesting detail: in the conflict scenario, the search returned a planted note telling the agent to ignore its instructions and reserve everything. The agent never attempted a reservation.

These results have limits. The model's decisions are scripted, so the benchmark really tests how my code handles fixed sequences, not how a real AI would behave. Also, none of the blocked scenarios ever tried to reserve, so the action gate never had to deny anything here. The 0% unsafe attempt rate just means the scripts behaved well. My own tests are where the gate actually gets tested.

3. Student Tests

I wrote four tests in tests/test_student_cases.py. Each one uses ScriptedLLM to make the model do something wrong on purpose, then checks the trace, the lab's state, and the final response.

Test 1 (unsafe action denied): Bob doesn't have laser safety training, but he goes through all the checks and then tries to book the laser cutter. The test confirms that the gate denied the reservation because of the missing qualification and that no new reservation appeared in the lab. Test 2 (contradictory final response): Alice tries to book a time that overlaps an existing reservation. The gate blocks it, but the

model still says the booking was completed. The test confirms the run is marked as failed and that no completed actions are reported. Interestingly, the lab simulator never sees this overlap: the gate catches it first, because the status check already showed the machine as unavailable. Test 3 (malformed output): the model returns a decision with an extra field that isn't allowed, so the run stops immediately and nothing gets executed. Test 4 (identity swap): while helping Alice, the model tries to look up Carol's authorizations. The gate blocks it and no data is returned, which shows the user check protects lookups too, not just reservations.

Together, these cover the main ways the model could misbehave: trying something unsafe, lying about the result, sending broken output, or acting as the wrong person.

4. Implementation Decisions

Error codes. My validators return short error codes like project_id_does_not_match_request instead of full sentences. This made the tests easier to write, since I could check for the exact reason something failed. The precondition check lists every problem at once instead of stopping at the first one, so the model knows everything that needs to change.

Denied calls. When the gate blocks a call, I still save it in the trace and send it back to the model as a normal failed result with the error code action_gate_denied. That way the trace shows everything the model tried, and the model can respond to the denial instead of guessing.

Termination reasons. I used five: model_finished, clarification_required, invalid_model_decision, invalid_final_response, and maximum_steps_exceeded. Keeping them separate makes it clear why each run ended.

Missing request details. The user ID always has to match the logged-in user, for every tool. For the project, resource, and times, if the request already has a value, the model must use that exact value. If the request left one blank, the model can still use it for a lookup, but not for a reservation, since that would mean guessing on the user's behalf.

Limitations. These checks only cover the specific rules I wrote, so they don't prove the system is safe in every situation. For example, nothing in my validators stops the model from making a second reservation after the first one succeeds. The prompt tells it not to, but the prompt is only a suggestion. The real protection comes from the checks in the code.

5. Data Flow Chart

